

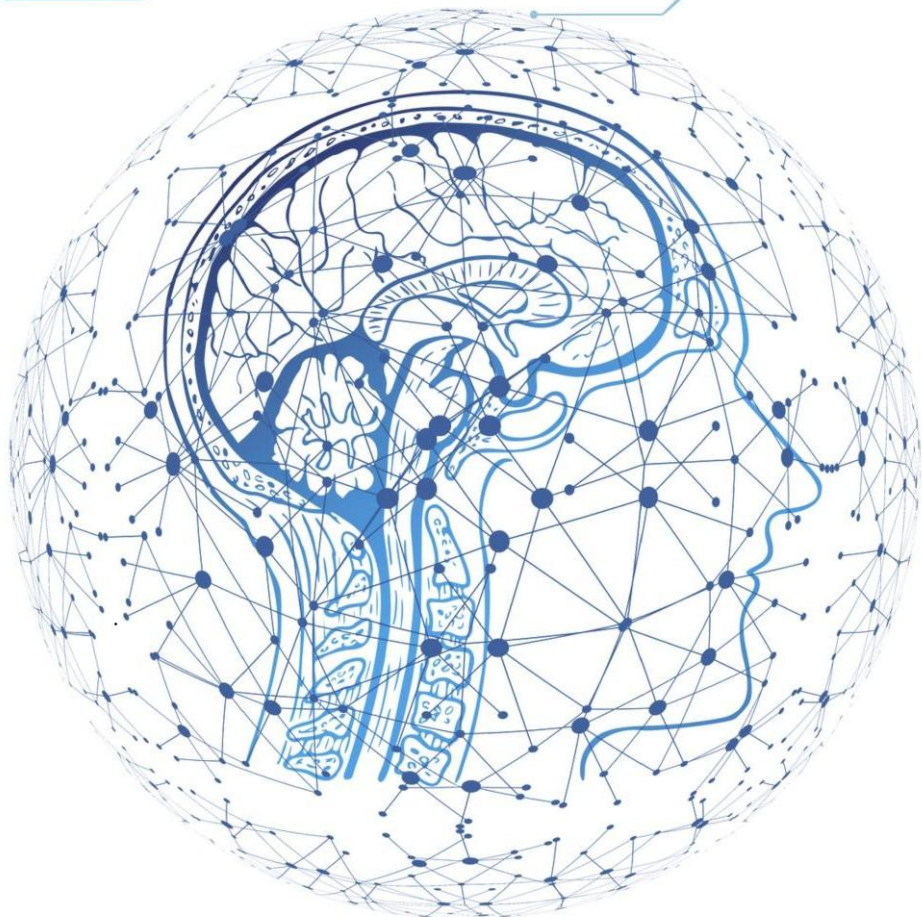
DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

www.hoasen.edu.vn

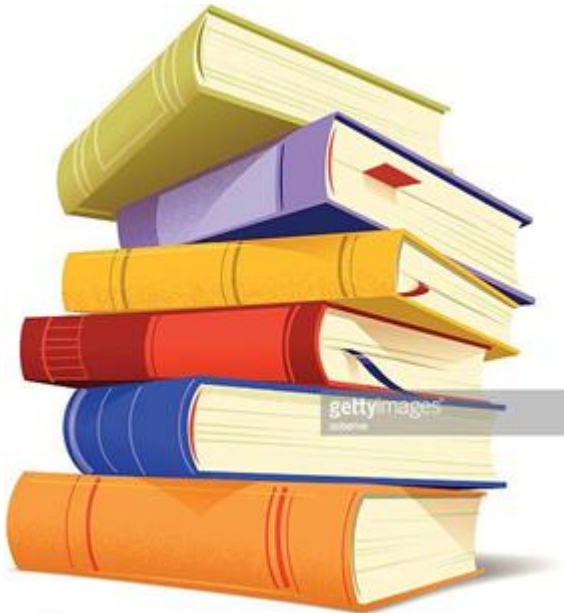
Chương 11

Deep learning for text Học sâu cho văn bản



Deep learning for text

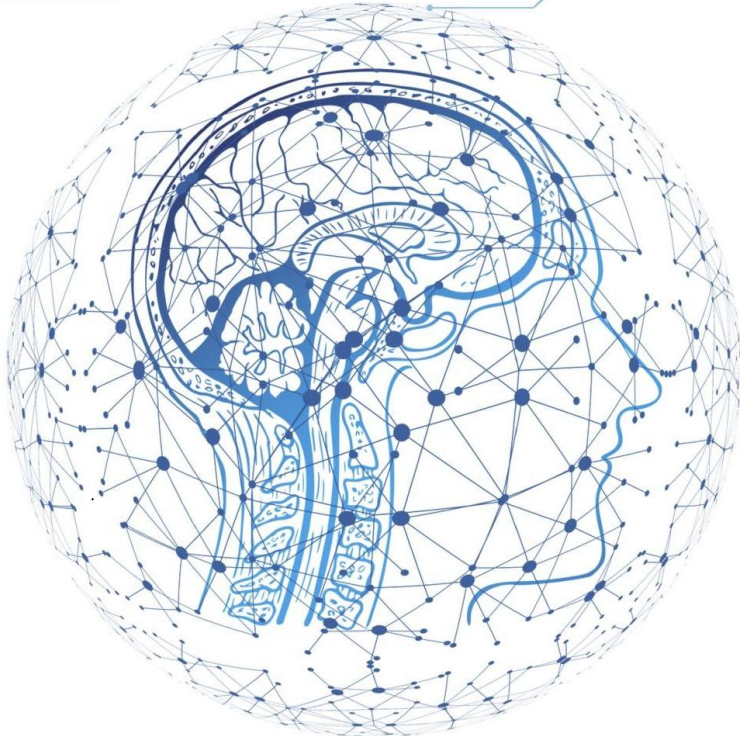
Học sâu cho văn bản



- NLP: Nhìn toàn cảnh
- Chuẩn bị dữ liệu văn bản
- Biểu diễn nhóm từ: Tập và chuỗi
- Kiến trúc Transformer
- Học sequence-to-sequence

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



NLP: The bird's eye view
NLP: Nhìn toàn cảnh

Natural language vs. machine language

Ngôn ngữ tự nhiên và ngôn ngữ máy

- **Ngôn ngữ máy** được thiết kế bằng luật rõ ràng: cú pháp, từ vựng, ý nghĩa.
- **Ngôn ngữ tự nhiên** hình thành từ cách con người sử dụng trước, luật xuất hiện sau.
- Văn bản tự nhiên thường **mơ hồ, lộn xộn**, thay đổi theo văn hóa và thời gian.
- Mục tiêu NLP: tạo thuật toán có thể xử lý văn bản, câu, đoạn và ý nghĩa tổng thể.

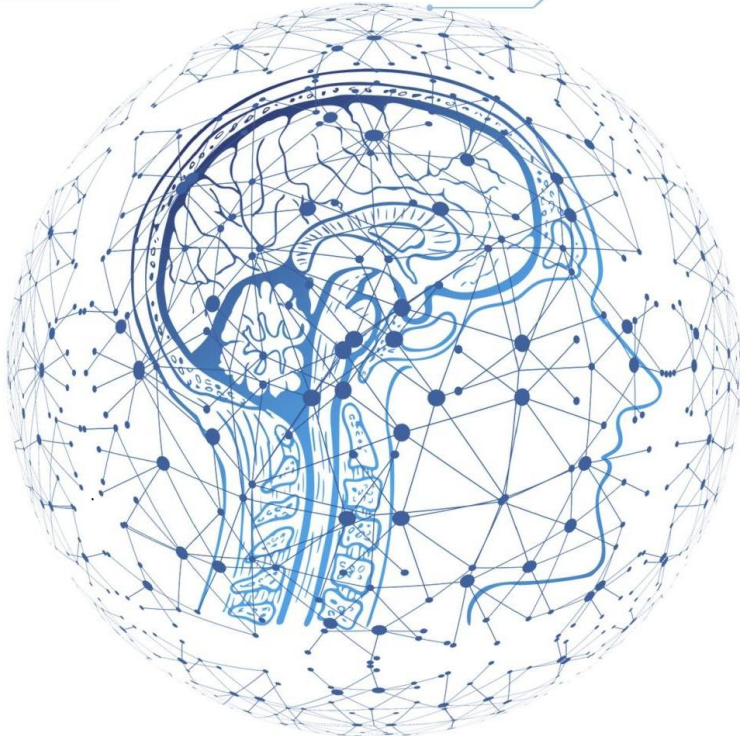
NLP as pattern recognition

NLP như nhận dạng mẫu

- Mô hình NLP hiện đại chủ yếu học **quy luật thống kê** trong văn bản.
- Tương tự computer vision học mẫu từ **pixels**, NLP học mẫu từ **words, sentences, paragraphs**.
- Trước deep learning, NLP phụ thuộc nhiều vào **feature engineering** thủ công.
- Deep learning chuyển trọng tâm sang học biểu diễn: **embeddings, RNN, attention, Transformer**.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Preparing text data
Chuẩn bị dữ liệu văn bản

From raw text to vectors Từ văn bản thô đến vector

- Pipeline cơ bản gồm 4 bước: **standardization, tokenization, indexing, vector encoding**.
- Văn bản ban đầu là chuỗi ký tự; neural network chỉ nhận **tensor số**.
- Mỗi bước quyết định mô hình sẽ “nhìn thấy” thông tin nào và bỏ qua thông tin nào.
- Sai lệch nhỏ trong preprocessing có thể làm mô hình học khó hơn hoặc tổng quát kém hơn.

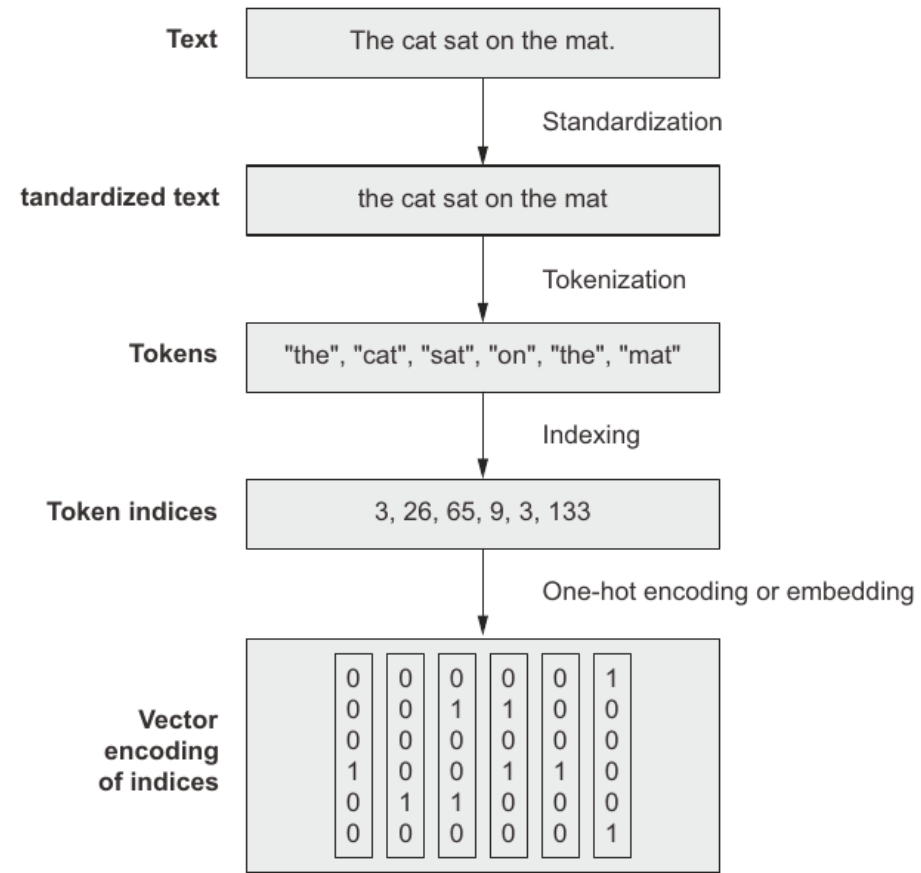


figure 11.1 From raw text to vectors

Hình 11.1. Từ văn bản thô đến vector.

Text standardization Chuẩn hóa văn bản

- **Standardization** xóa các khác biệt ký tự mà ta không muốn model phải tự học.
- Ví dụ phổ biến: chuyển sang chữ thường, bỏ punctuation: `Sunset?` → `sunset`.
- Có thể chuẩn hóa Unicode: `México` → `mexico`, `é` → `e`.
- **Stemming/Lemmatization** (Rút gọn từ/Phân tích từ gốc) gom biến thể của từ: `staring`, `stared` → gốc gần giống nhau.

Tokenization

Tách token

- **Tokenization** chia văn bản thành đơn vị rời rạc để mô hình xử lý.
- Word-level: `the cat sat` → `the`, `cat`, `sat`.
- N-gram: lấy nhóm token liên tiếp, ví dụ `the cat`, `cat sat`.
- Character-level hoặc subword-level hữu ích khi từ vựng lớn, nhiều từ hiếm hoặc nhiều lỗi chính tả.

Vocabulary indexing Lập chỉ mục từ vựng

- Sau tokenization, xây **vocabulary** từ tập train và gán mỗi token một số nguyên.
- Ví dụ: `the` → 3`, `cat` → 26`, `sat` → 65`.
- Thường giới hạn vocabulary ở **20,000-30,000 token phổ biến nhất**.
- Token hiếm hoặc chưa thấy trong train được ánh xạ thành **[UNK]**.

TextVectorization layer Layer TextVectorization

- Keras cung cấp **TextVectorization** để chuẩn hóa, token hóa, lập vocabulary và vector hóa.
- Mặc định: chuyển chữ thường, bỏ punctuation, tách theo khoảng trắng.
- Các `output\_mode` chính: `int`, `multi\_hot`, `count`, `tf\_idf`.
- Gọi `adapt(train\_texts)` để layer học vocabulary từ dữ liệu train.

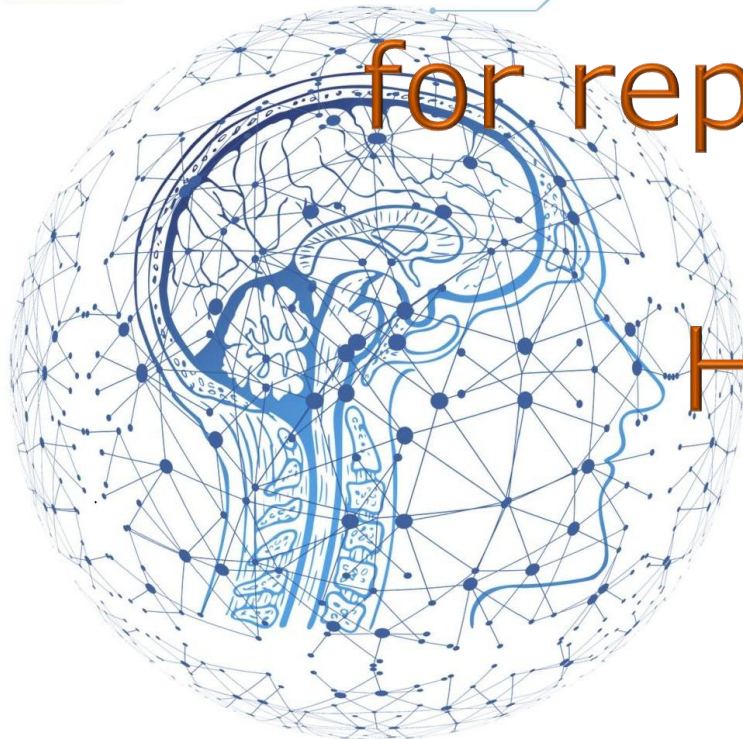
Encoding and decoding example

Ví dụ mã hóa và giải mã

- Câu test: `I write, rewrite, and still rewrite again`.
- Sau vectorization: một chuỗi số nguyên, ví dụ `[7, 3, 5, 9, 1, 5, 10]`.
- Khi giải mã lại, token ngoài vocabulary thành **[UNK]**.
- Thông điệp: preprocessing là một phần của mô hình hóa, không chỉ là bước kỹ thuật phụ.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Two approaches
for representing groups of words:
Sets and sequences
Hai cách biểu diễn nhóm từ:
Tập và chuỗi

Sets vs. sequences

Tập từ và chuỗi từ

- Câu hỏi trung tâm: mô hình có cần biết **thứ tự từ** không?
- **Bag-of-words** bỏ qua thứ tự, xem văn bản như một tập/histogram token (Phân bố tần suất token).
- **Sequence models** xử lý thứ tự: RNN, 1D convnet, Transformer.
- Trong phân loại văn bản, cả hai hướng vẫn còn hữu ích; Transformer không luôn là lựa chọn tốt nhất.
- Ví dụ: tôi thích học sâu vì học sâu rất thú vị
- Tập từ vựng: ["tôi", "thích", "học", "sâu", "vì", "rất", "thú", "vị"]
- Histogram được biểu diễn bằng vector: [1, 1, 2, 2, 1, 1, 1, 1]

IMDB raw movie reviews

Dữ liệu review phim IMDB thô

- Dataset gồm **25,000 review train** và **25,000 review test**, cân bằng positive/negative.
- Mỗi review là một file text trong thư mục `pos` hoặc `neg`.
- Tách validation bằng cách lấy **20%** từ train.
- Dùng `text\_dataset\_from\_directory()` để tạo `tf.data.Dataset` từ thư mục.

Bag-of-words with unigrams

Bag-of-words với unigram

- Với unigram, câu `the cat sat on the mat` thành tập token riêng lẻ.
- Vector **multi-hot** có kích thước bằng vocabulary: token xuất hiện $\rightarrow 1$, không xuất hiện $\rightarrow 0$.

`TextVectorization(max_tokens=20000, output_mode="multi_hot")`

- Trên IMDB, model Dense nhỏ đạt khoảng **89.2% test accuracy**.

Bigrams and local order Bigram và thứ tự cục bộ

- **Bigram** là cặp token liên tiếp: `the cat`, `cat sat`, `sat on`.
- N-gram đưa lại một phần **local order information** cho bag-of-words.

`TextVectorization(ngrams=2, output_mode="multi_hot")`

- Trên IMDB, bag-of-bigrams đạt khoảng **90.4% test accuracy**, tốt hơn unigram.

TF-IDF weighting Trọng số TF-IDF

- **Term frequency:** token xuất hiện nhiều trong document thì có thể quan trọng hơn.
- **Document frequency:** token xuất hiện ở hầu hết document thường ít phân biệt hơn.
- Công thức ý tưởng:
$$tfidf = tf \times \log(N/df)$$
- Trong sách, TF-IDF bigram đạt khoảng **89.8%** trên IMDB, không hơn binary bigram ở ví dụ này.

IMDB text-classification results

Kết quả phân loại IMDB

- **Naive baseline:** khoảng 50% vì dữ liệu cân bằng hai lớp.
- Binary unigram: khoảng **89.2%** test accuracy.
- Binary bigram: khoảng **90.4%**, tốt nhất trong các thử nghiệm ban đầu.
- TF-IDF bigram: khoảng **89.8%** trong ví dụ sách.
- Bài học: phương pháp đơn giản vẫn có thể rất mạnh khi phù hợp dữ liệu.

Sequence model approach Cách tiếp cận sequence model

- Đầu vào là chuỗi số nguyên: $x = [x_1, x_2, \dots, x_T]$
- Mỗi index được ánh xạ thành vector, tạo chuỗi vector.
- Layer phía sau học tương tác theo thứ tự: **RNN, 1D convnet**, hoặc **Transformer**.
- Ý tưởng: để model tự học feature theo thứ tự thay vì thiết kế n-gram thủ công.

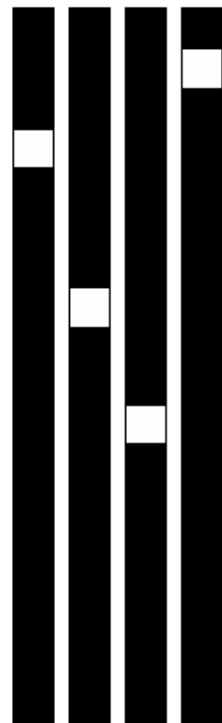
Why one-hot sequences are costly Vì sao one-hot sequence đắt

- Nếu review dài 600 token và vocabulary 20,000 token: input có kích thước **600 × 20,000**.
- Một review cần khoảng **12,000,000 float** nếu one-hot toàn bộ sequence.
- Mô hình Bidirectional LSTM trên one-hot chạy chậm và chỉ đạt khoảng **87%**.
- Cần biểu diễn tốt hơn: **word embeddings**.

Word embeddings

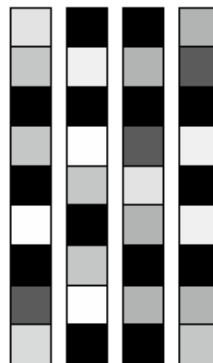
Vector nhúng từ

- **Embedding** là vector dense, số chiều thấp hơn nhiều so với one-hot.
- Các từ gần nghĩa có xu hướng nằm gần nhau trong embedding space.
- Hướng trong không gian có thể mang ý nghĩa, ví dụ **gender**, **plural**, hoặc quan hệ ngữ nghĩa.
- Embedding có thể **học từ task hiện tại** hoặc dùng **pretrained embeddings**.



One-hot word vectors:

- Sparse
- High-dimensional
- Hardcoded



Word embeddings:

- Dense
- Lower-dimensional
- Learned from data

Figure 11.2 Word representations obtained from one-hot encoding or hashing are sparse, high-dimensional and hardcoded. Word embeddings are dense, relatively low-dimensional, and learned from data.

Hình 11.2. One-hot sparse, embedding dense và học được từ dữ liệu.

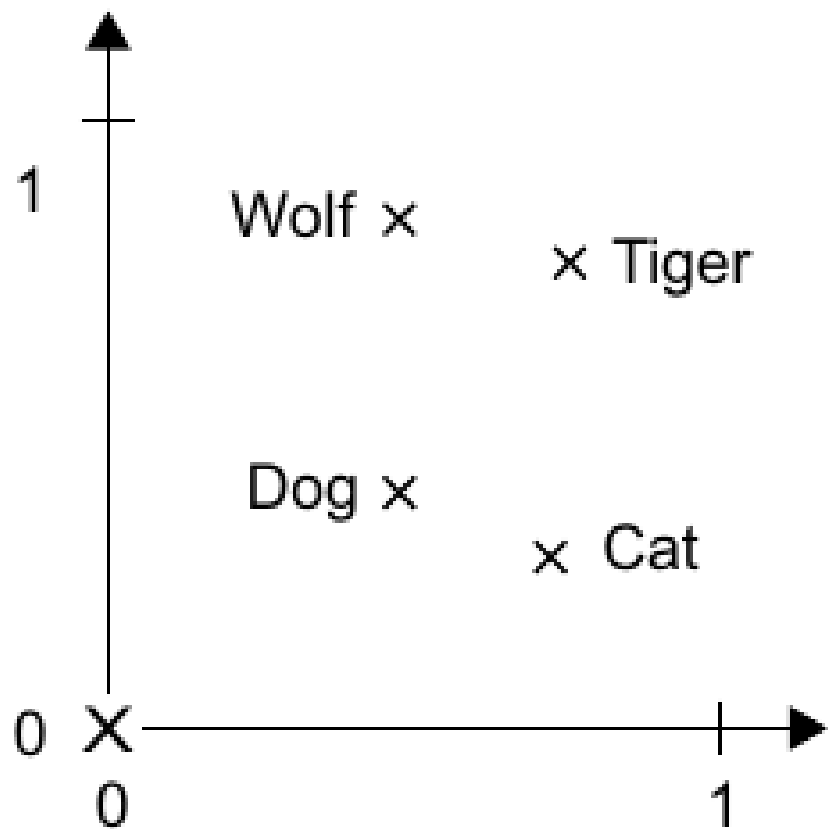


Figure 11.3 A toy example of a word-embedding space

Hình 11.3. Ví dụ đồ chơi về không gian embedding.

Embedding layer

Layer Embedding

- ``Embedding(input_dim=max_tokens, output_dim=256)`` tạo bảng tra cứu vector.
- Input shape: ``(batch_size, sequence_length)`` gồm integer token indices.
- Output shape: ``(batch_size, sequence_length, embedding_dim)`` gồm vector thực.
- Weights ban đầu random, sau đó được cập nhật bằng **backpropagation**.

Word index → Embedding layer → Corresponding word vector

Figure 11.4 The Embedding layer

Hình 11.4. Layer Embedding như phép tra cứu vector.

Padding and masking

Padding và masking

- Các câu có độ dài khác nhau, nhưng batch cần tensor có cùng chiều dài.
- **Padding** thêm token 0 vào cuối sequence ngắn.
- RNN có thể bị nhiễu nếu đọc nhiều bước padding vô nghĩa.
- ``Embedding(..., mask_zero=True)`` tạo mask để layer sau bỏ qua padding.

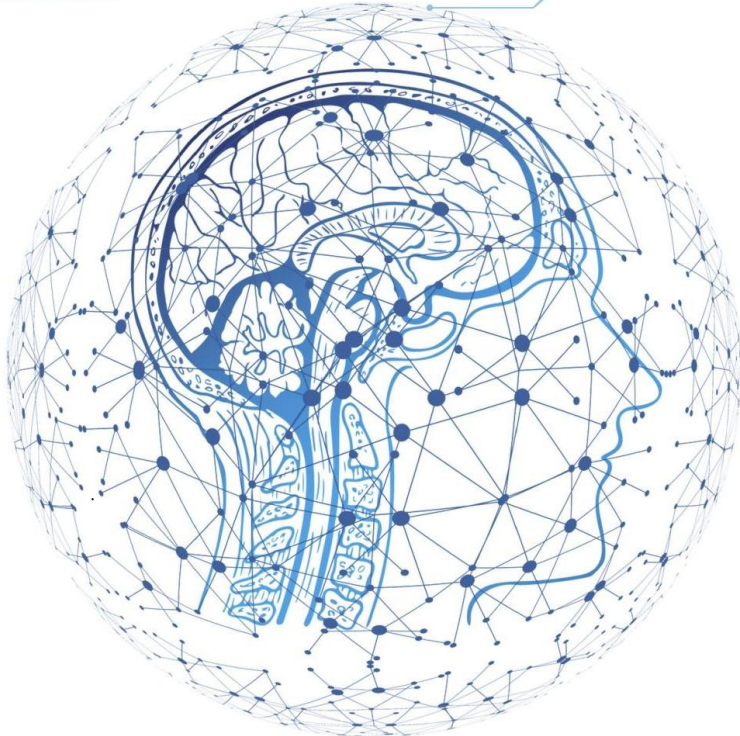
Pretrained word embeddings Embedding được huấn luyện sẵn

- **Pretrained embeddings** học từ corpus lớn bằng task khác, ví dụ GloVe.
- Hữu ích khi dataset hiện tại **nhỏ**, không đủ để tự học embedding mạnh.
- Trong sách dùng GloVe 100D từ English Wikipedia 2014.
- Với IMDB, pretrained embeddings không giúp nhiều vì tập train đủ lớn để học embedding chuyên biệt.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

The Transformer architecture Kiến trúc Transformer



Why Transformer matters

Vì sao Transformer quan trọng

- Từ 2017, **Transformer** thay thế RNN trong nhiều task NLP.
- Ý tưởng cốt lõi từ paper **Attention is all you need**: dùng attention để mô hình hóa sequence.
- Transformer xử lý token song song tốt hơn RNN và học tương tác xa hiệu quả hơn.
- Trong chương này, ta xây **TransformerEncoder** cho phân loại IMDB và **TransformerDecoder** cho dịch máy.

Attention idea Ý tưởng attention

- **Attention** gán điểm quan trọng cho các feature đầu vào.
- Feature quan trọng được tăng ảnh hưởng; feature ít liên quan bị giảm ảnh hưởng.
- Max pooling và TF-IDF đều có thể xem như trực giác attention đơn giản.
- Attention hiện đại còn giúp tạo representation **phụ thuộc ngữ cảnh**.

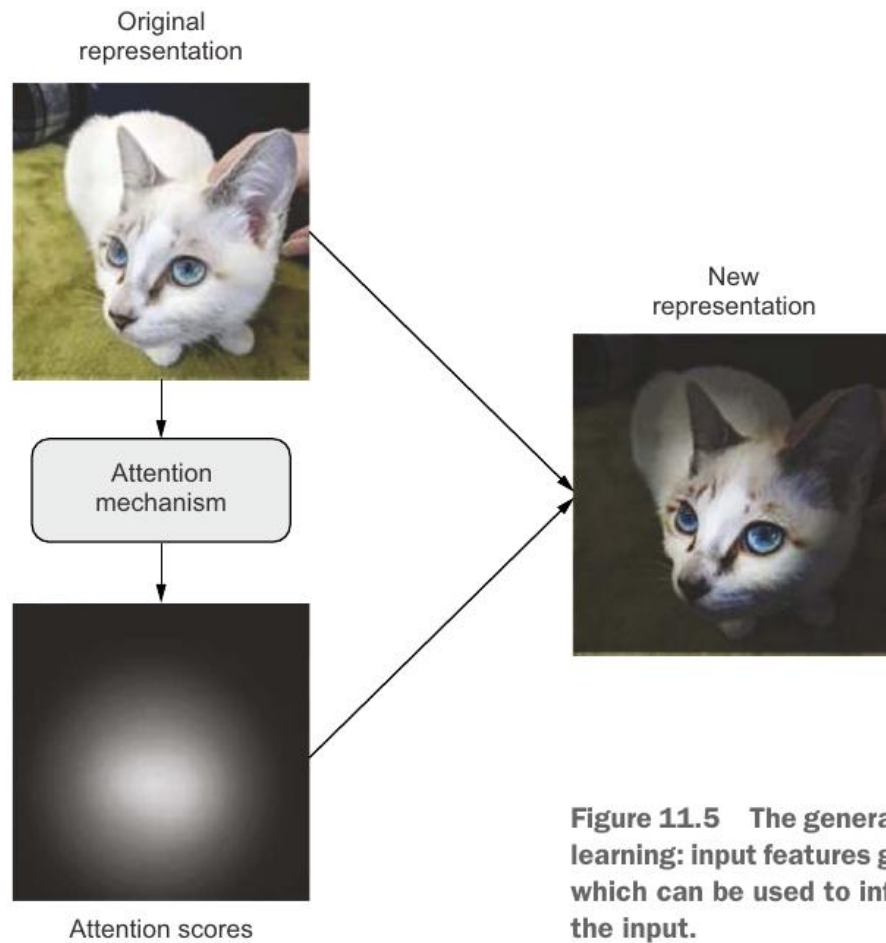


Figure 11.5 The general concept of “attention” in deep learning: input features get assigned “attention scores, which can be used to inform the next representation of the input.

Hình 11.5. Khái niệm chung của attention trong deep learning.

- **Self-attention** làm mỗi token nhìn các token khác trong cùng sequence.
- Mục tiêu: biến vector cố định của từ thành vector **context-aware**.
- Ví dụ `station` trong `train station` khác `radio station`.
- Điểm tương quan đơn giản: $s_{ij} = x_i^T x_j$; sau đó softmax và weighted sum.

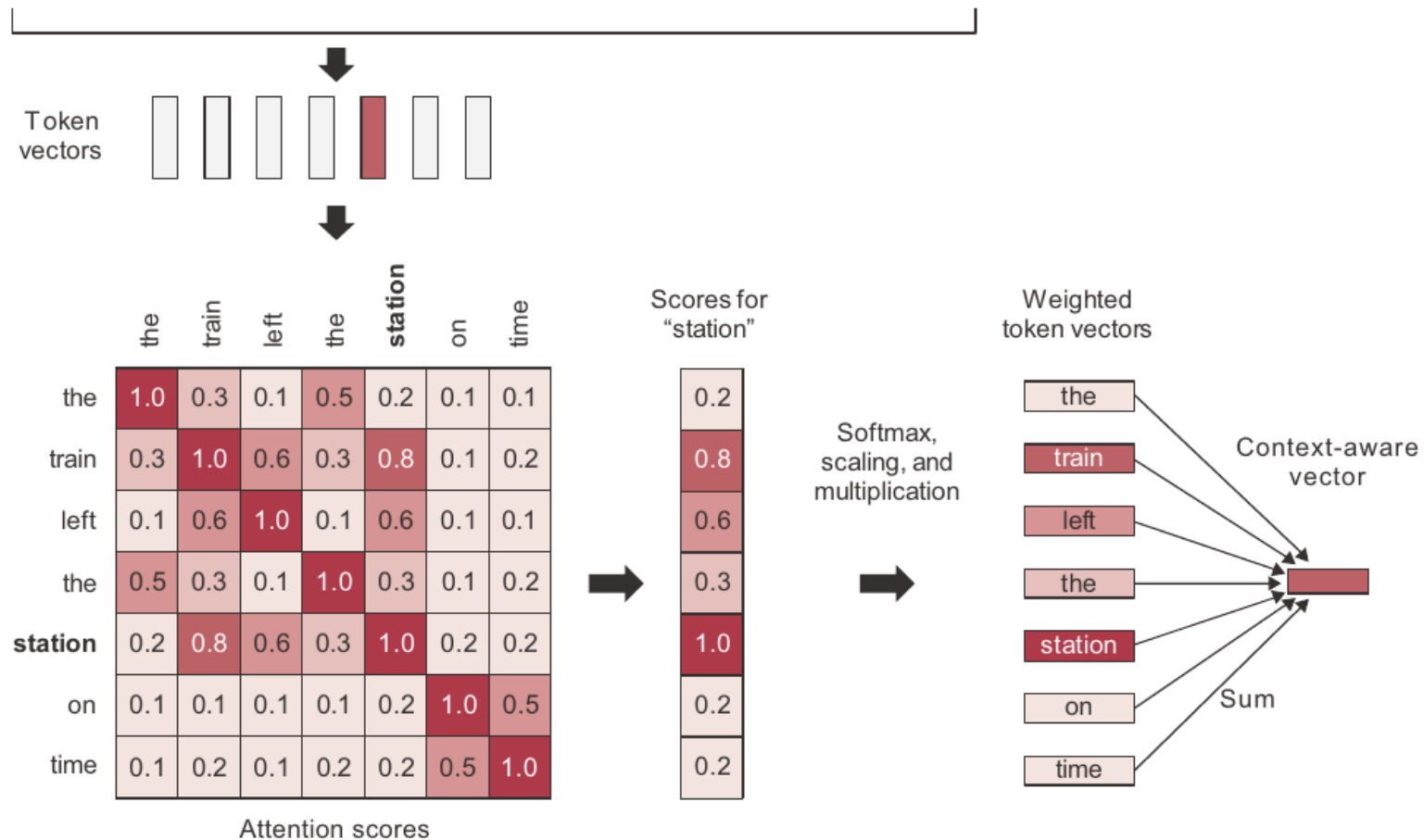


Figure 11.6 Self-attention: attention scores are computed between "station" and every other word in the

Hình 11.6. Self-attention tạo vector phụ thuộc ngữ cảnh.

Query-Key-Value model

Mô hình Query-Key-Value

- Attention tổng quát dùng ba tensor: **Query (Q), Key (K), Value (V)**.
- Với mỗi query, so khớp với các key để lấy attention scores.
- Scores dùng để trộn các value thành output.
- Công thức phổ biến:

$$A(Q, K, V) = \text{softmax}\left(QK^T / \sqrt{d_k}\right)V$$

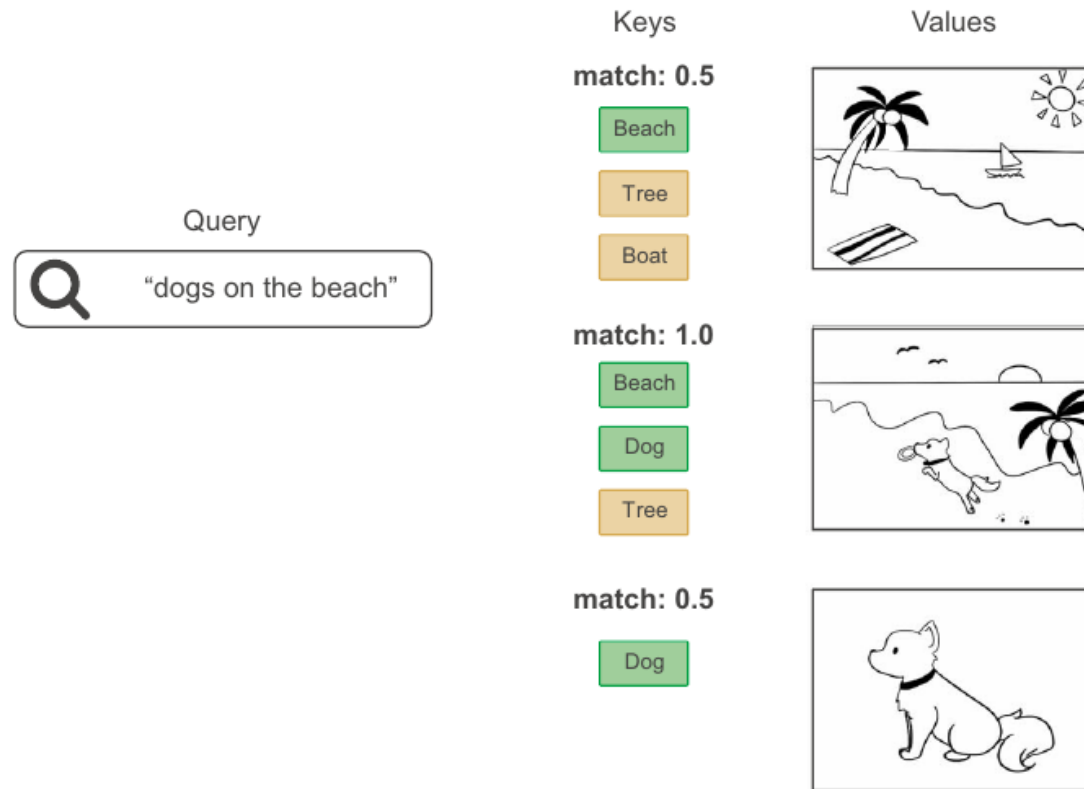
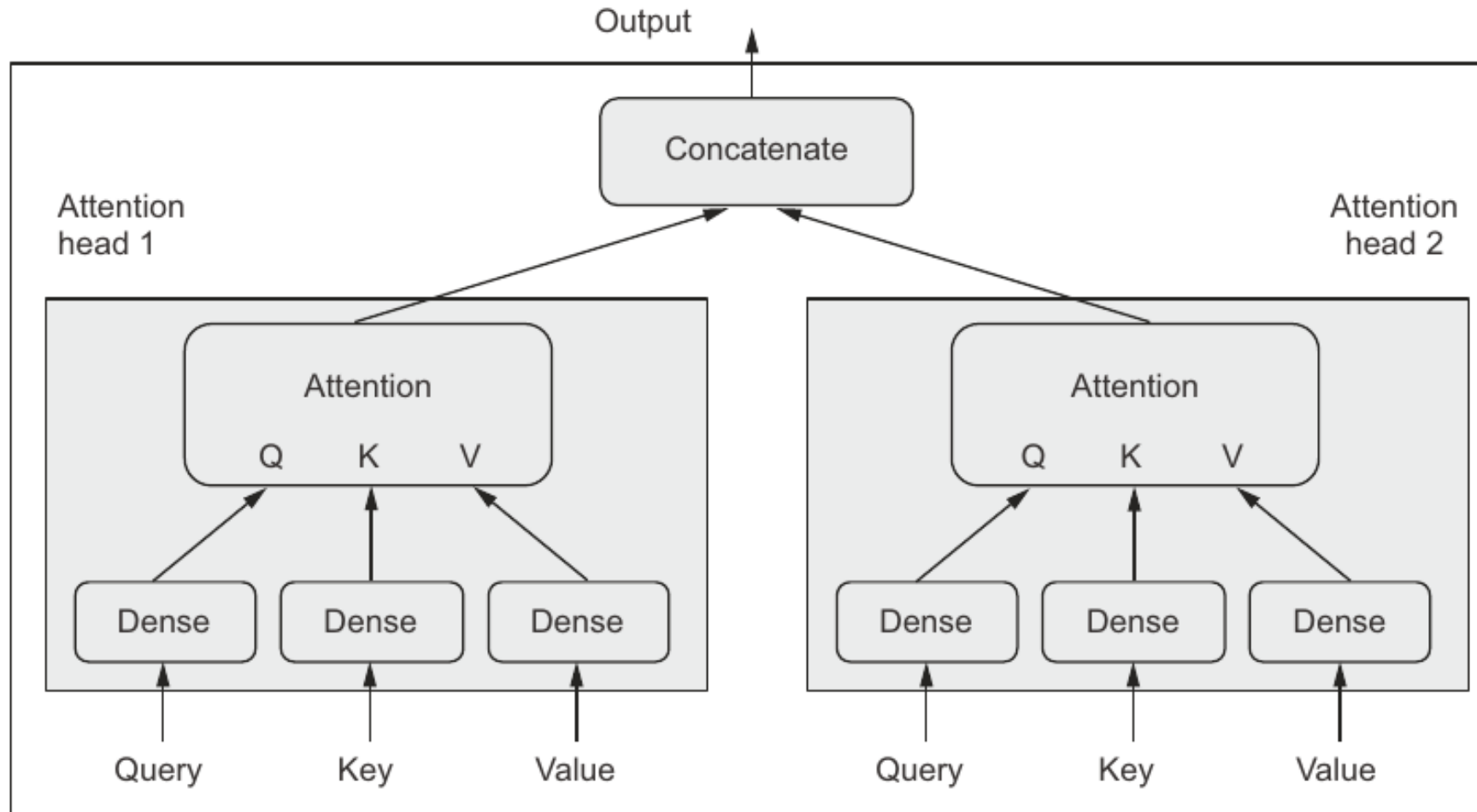


Figure 11.7 Retrieving images from a database: the “query” is compared to a set of “keys,” and the match scores are used to rank “values” (images).

Hình 11.7. Query so khớp với Keys để xếp hạng Values.

Multi-head attention Attention nhiều head

- **Multi-head attention** chia không gian output thành nhiều subspace học độc lập.
- Mỗi head có projection riêng cho Q, K, V và học một kiểu quan hệ khác nhau.
- Outputs của các head được **concatenate** rồi projection tiếp.
- Nhiều head giúp model quan sát nhiều nhóm đặc trưng cùng lúc.



Hình 11.8. Layer MultiHeadAttention.

Transformer encoder block Khối Transformer encoder

- Encoder block gồm **MultiHeadAttention, Dense projection, residual connections, LayerNormalization**.
- Residual connections giúp giữ thông tin và hỗ trợ gradient trong mô hình sâu.
- LayerNormalization phù hợp sequence hơn BatchNormalization.
- Encoder biến chuỗi token vectors thành chuỗi representation hữu ích hơn.

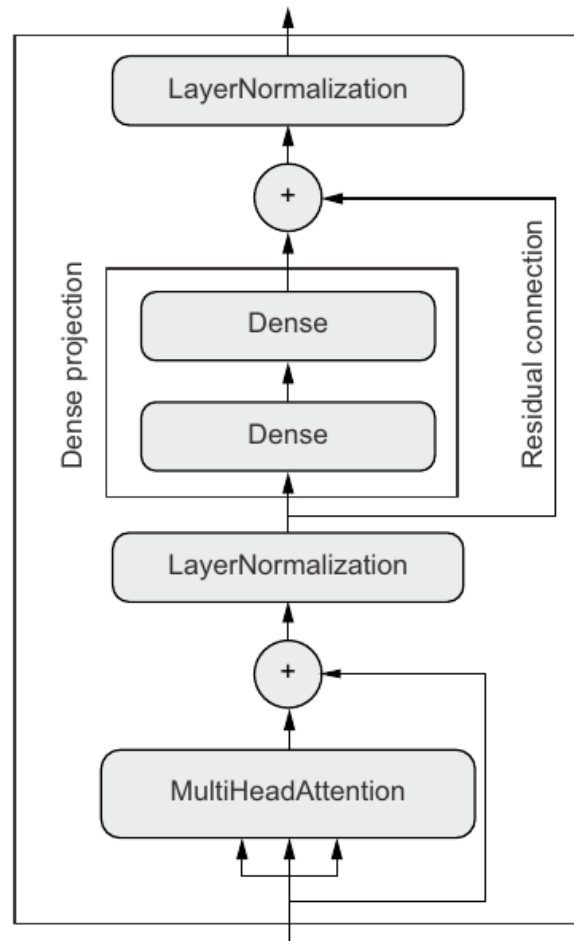


Figure 11.9 The TransformerEncoder chains a MultiHeadAttention layer with a dense projection and adds normalization as well as residual connections.

Hình 11.9. TransformerEncoder gồm attention, dense projection và normalization.

LayerNormalization vs. BatchNormalization

LayerNorm và BatchNorm

- **BatchNormalization** lấy thống kê qua batch và thường dùng tốt trong computer vision.
- **LayerNormalization** chuẩn hóa từng sequence/sample riêng trên trục feature.
- Công thức ý tưởng: $\hat{x} = \frac{x - \mu}{\sigma}$ trên feature axis của từng mẫu.
- Với sequence data, LayerNorm thường ổn định hơn BatchNorm.

Positional embedding Embedding vị trí

- Self-attention tự thân là cơ chế xử lý **set**, chưa biết thứ tự token.
- Để Transformer thành sequence model, cần thêm **position information**.
- Cách làm: học vector vị trí và cộng với word embedding.
- Công thức: $z_i = w_i + p_i$, với w_i là word embedding và p_i là positional embedding.

	Word order awareness	Context awareness (cross-words interactions)
Bag-of-unigrams	No	No
Bag-of-bigrams	Very limited	No
RNN	Yes	No
Self-attention	No	Yes
Transformer	Yes	Yes

Figure 11.10 Features of different types of NLP models

Hình 11.10. So sánh word order và context awareness của các mô hình NLP.

Text-classification Transformer

Transformer cho phân loại văn bản

- Model: **PositionalEmbedding** → **TransformerEncoder** → **GlobalMaxPooling1D** → **Dropout** → **Dense**.
- Không có positional embedding, Transformer encoder đạt khoảng **87.5%** trên IMDB.
- Có positional embedding, kết quả tăng lên khoảng **88.3%**.
- Vẫn thấp hơn **bag-of-bigrams 90.4%** trong ví dụ này.

When to use sequence models Khi nào dùng sequence model

- Rule of thumb trong sách: xét tỷ lệ $N_{samples}/L_{mean}$
- Nếu tỷ lệ $< \mathbf{1500}$: thường chọn **bag-of-bigrams** cho text classification.
- Nếu tỷ lệ $> \mathbf{1500}$: thường chọn **sequence model**.
- Sequence model hợp hơn khi có **nhieu dữ liệu** và mỗi sample **ngắn**.

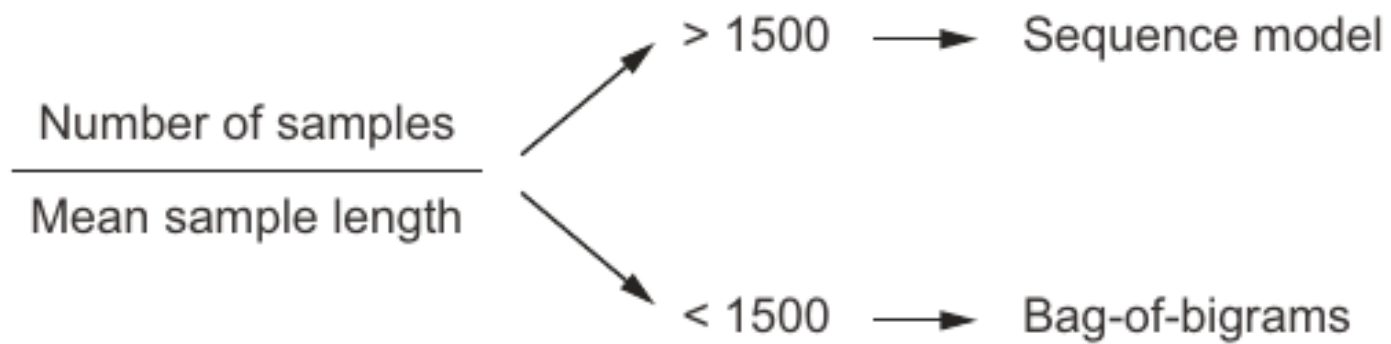


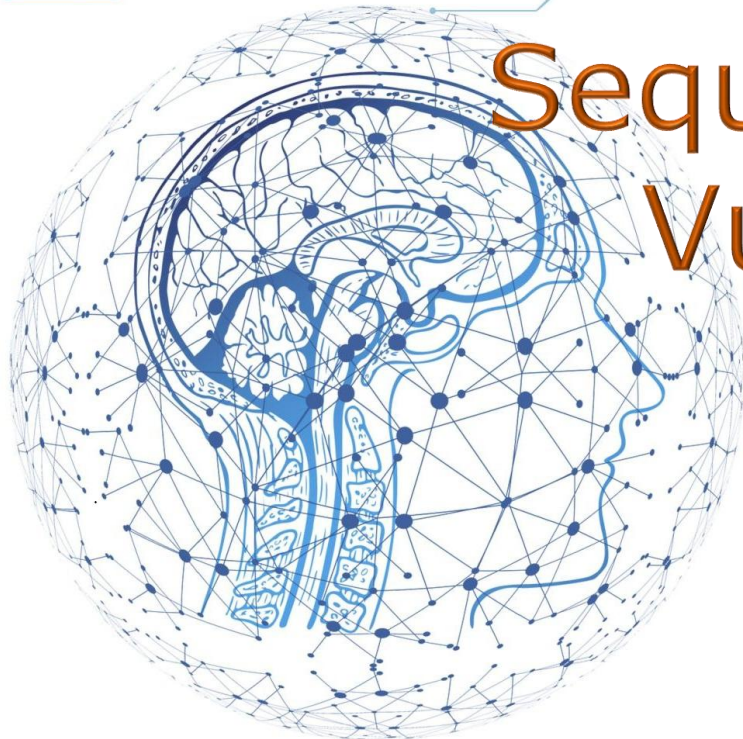
Figure 11.11 A simple heuristic selecting a text-classification model based on the ratio between the number of training samples and the mean number of words per sample

Hình 11.11. Heuristic chọn mô hình phân loại văn bản.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Beyond text classification:
Sequence-to-sequence learning
Vượt khỏi phân loại văn bản:
Học chuỗi-sang-chuỗi



Beyond classification

Không chỉ phân loại văn bản

- **Sequence-to-sequence** nhận một sequence và sinh sequence khác.
- Ứng dụng: **machine translation**, summarization, question answering, chatbot, text generation.
- Mẫu chung: **encoder** xử lý source sequence, **decoder** sinh target sequence.
- Trong inference, target được sinh **từng token một** cho tới token kết thúc.

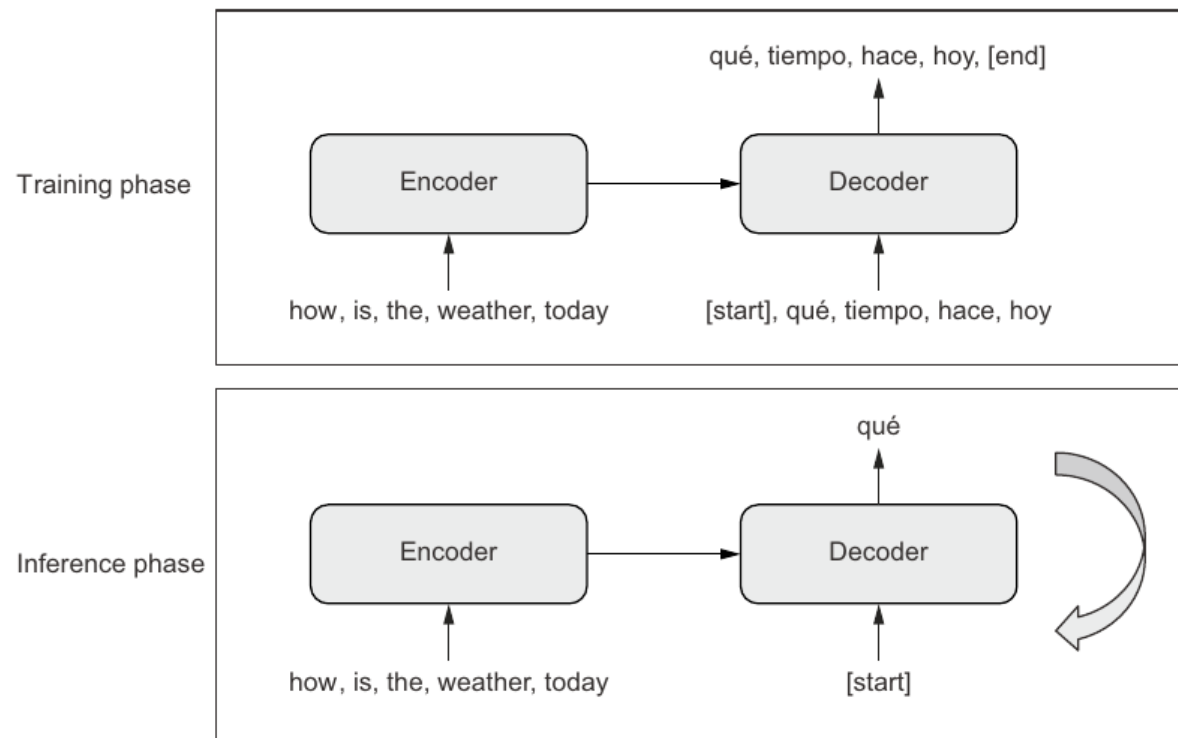


Figure 11.12 Sequence-to-sequence learning: the source sequence is processed by the encoder and is then sent to the decoder. The decoder looks at the target sequence so far and predicts the target sequence offset by one step in the future. During inference, we generate one target token at a time and feed it back into the decoder.

Hình 11.12. Template sequence-to-sequence: training và inference.

Machine translation example

Ví dụ dịch máy

- Sách dùng dataset English-Spanish từ ``manythings.org/anki``.
- Mỗi dòng: English sentence + tab + Spanish sentence.
- Thêm token ``[start]`` và ``[end]`` vào câu Spanish.
- Dùng hai **TextVectorization** layer riêng cho source English và target Spanish.

Preparing decoder inputs Chuẩn bị input cho decoder

- Target Spanish được vector hóa dài hơn 1 token để tạo cặp input/target lệch nhau.
- ``decoder_inputs = spanish[:, :-1]`` : bỏ token cuối.
- ``targets = spanish[:, 1:]`` : bỏ token đầu, tức target đi trước một bước.
- Ý tưởng huấn luyện: dự đoán token t_{i+1} từ các token $t_0 \dots t_i$ và source encoded.

RNN sequence-to-sequence Seq2seq bằng RNN

- Encoder RNN đọc source và tạo vector/state biểu diễn toàn bộ câu nguồn.
- Decoder RNN nhận state đó làm initial state và sinh target sequence.
- Sách dùng **GRU** để đơn giản vì GRU có một state vector.
- Mô hình RNN toy đạt khoảng **64% next-token accuracy**.

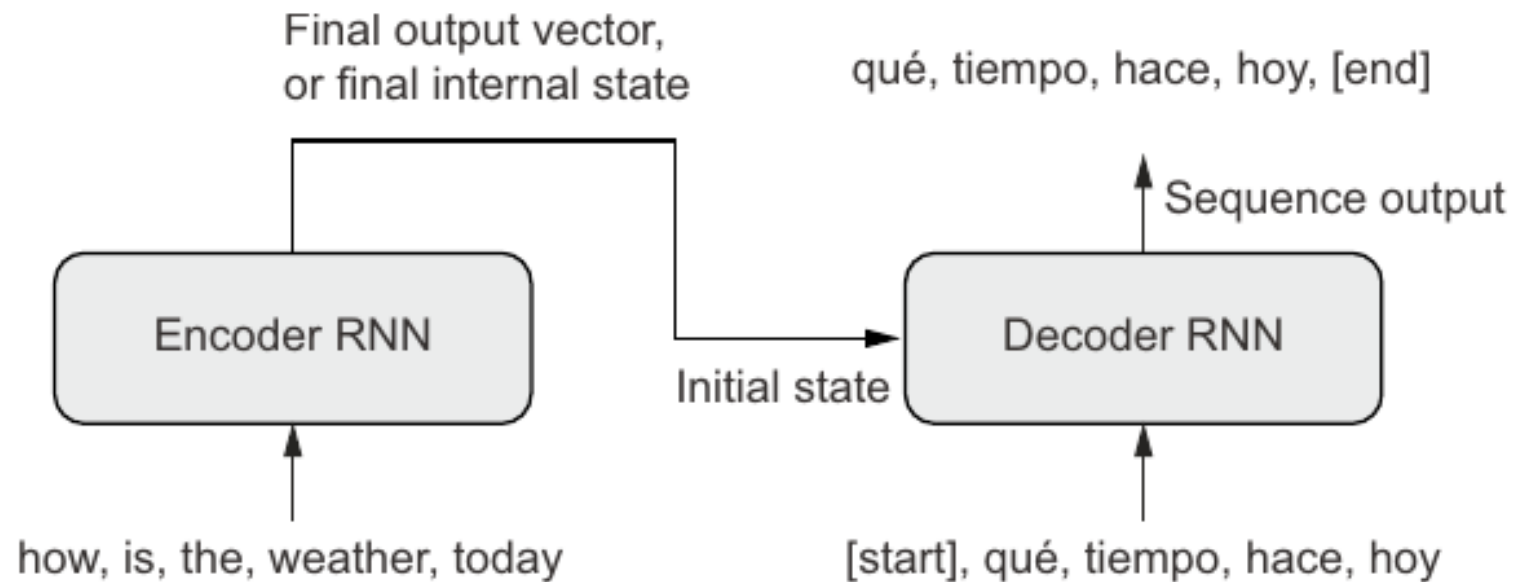


Figure 11.13 A sequence-to-sequence RNN: an RNN encoder is used to produce a vector that encodes the entire source sequence, which is used as initial state for an RNN decoder.

Hình 11.13. Sequence-to-sequence RNN với encoder và decoder.

Limits of RNN seq2seq

Giới hạn của RNN seq2seq

- RNN xử lý tuần tự nên khó song song hóa và chậm với sequence dài.
- Encoder phải nén toàn bộ source vào một state, dễ mất context xa.
- Khi inference, token sai sớm có thể kéo theo lỗi ở các token sau.
- Các giới hạn này thúc đẩy việc dùng **Transformer** cho seq2seq.

Transformer seq2seq Seq2seq bằng Transformer

- Transformer encoder tạo representation context-aware cho source sequence.
- Transformer decoder sinh target và có thêm attention block nhìn sang **encoded source**.
- Decoder dùng **self-attention** trên target đã sinh và **cross-attention** tới source.
- Trong sách, Transformer toy đạt khoảng **67% accuracy**, cao hơn GRU-based model.

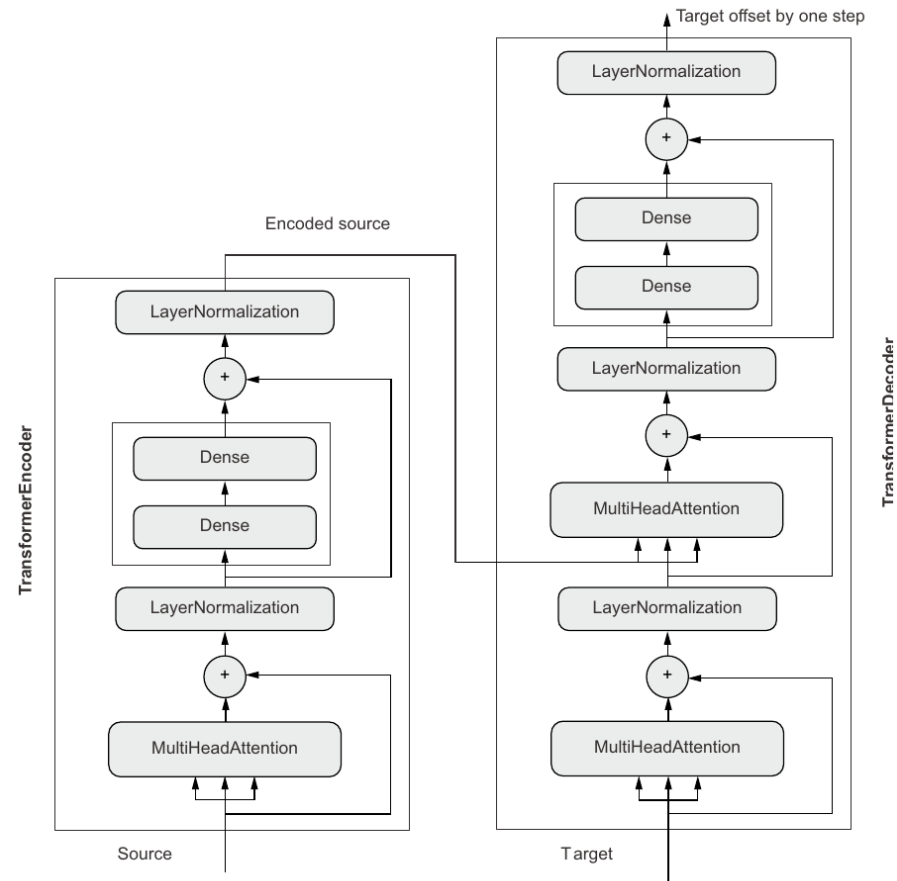


Figure 11.14 The TransformerDecoder is similar to the TransformerEncoder, except it features an additional attention block where the keys and values are the source sequence encoded by the TransformerEncoder. Together, the encoder and the decoder form an end-to-end Transformer.

Hình 11.14. TransformerDecoder trong mô hình Transformer end-to-end.

Causal mask in the decoder

Causal mask trong decoder

- Transformer decoder nhìn toàn bộ target sequence cùng lúc khi train.
- Nếu không mask, model có thể nhìn token tương lai và học **copy gian lận**.
- **Causal mask** chặn attention tới vị trí tương lai.
- Ý tưởng mask:

$$M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

- Trong đó:
 - i : vị trí token hiện tại đang dự đoán.
 - j : vị trí token mà model muốn nhìn vào.
 - M_{ij} : giá trị mask tại hàng i , cột j .
- Ý nghĩa:
 - Nếu $j \leq i$: token hiện tại được phép nhìn các token trước đó và chính nó.
 - Nếu $j > i$: token hiện tại không được nhìn token tương lai.

Inference and sample translations

Suy luận và ví dụ dịch

- Inference bắt đầu với `[start]`, mỗi vòng sinh một token bằng `argmax`.
- Token mới được nối vào decoded sentence rồi đưa lại vào decoder.
- Dừng khi sinh `[end]` hoặc đạt độ dài tối đa.
- Kết quả Transformer trong sách tốt hơn RNN toy, nhưng vẫn có lỗi dịch cơ bản.

Chapter Summary

Tổng kết chương

- NLP bắt đầu bằng biến **raw text** thành tensor qua standardization, tokenization, indexing và vectorization.
- Có hai họ mô hình chính: **bag-of-words** và **sequence models**.
- **Embeddings** biểu diễn từ bằng vector dense có cấu trúc ngữ nghĩa.
- **Attention** tạo representation phụ thuộc ngữ cảnh; Transformer cần thêm positional information để biết thứ tự.
- **Sequence-to-sequence** dùng encoder-decoder để sinh chuỗi mới, đặc biệt mạnh với Transformer.

